

Projet Blockchain

Jeu Devine Nombre

Cours : Blockchain

Université Paris-Saclay

Auteurs

Bedjou Celina
Ania Chikh

Table des matières

1	Introduction	2
2	Version 1 : Jeu de base	2
3	Version 2 : Jeu avec récompense en ETH	2
4	Version 3 : Smart Contract générique	3
5	Version 4 : Confidentialité du nombre (Commit–Reveal)	3
5.1	Commit	3
5.2	Reveal	3
6	Architecture de l’application	4

30 mars 2026

1 Introduction

Dans le cadre du cours de Blockchain, nous avons développé une application décentralisée (DApp) implémentant le jeu *Devine Nombre*. Dans ce jeu, le joueur A choisit un nombre secret et le joueur B tente de le deviner en plusieurs tentatives. Le projet utilise un *smart contract* sur la blockchain Ethereum afin de garantir l'exécution transparente et automatique des règles du jeu. L'application s'appuie sur plusieurs technologies : Solidity pour le smart contract, Python avec `web3.py` pour interagir avec la blockchain, Streamlit pour l'interface utilisateur et Remix pour le développement et le déploiement sur le réseau de test Ethereum Sepolia.

2 Version 1 : Jeu de base

La première version correspond à une implémentation simple du jeu. Le joueur A choisit un nombre secret entre 1 et 100 lors du déploiement du smart contract, tandis que le joueur B tente de le deviner. Le contrat maintient les informations essentielles du jeu : le nombre secret, le nombre maximal de tentatives, le nombre d'essais effectués et l'état de la partie. Le joueur B propose des nombres via la fonction `guess()`. Le smart contract compare la proposition avec le nombre secret et indique si la valeur est trop grande, trop petite ou correcte. La partie se termine lorsque le nombre est trouvé ou lorsque le nombre maximal de tentatives est atteint.

3 Version 2 : Jeu avec récompense en ETH

La deuxième version ajoute une dimension économique au jeu.

Dans cette version, les joueurs doivent miser de l'ETH pour participer. Le smart contract agit alors comme un arbitre financier qui conserve les fonds et les redistribue automatiquement au gagnant.

Le déroulement du jeu est le suivant :

1. Le joueur A crée la partie et envoie une mise en ETH.
2. Le joueur B rejoint la partie et dépose également une mise.
3. Le joueur B tente de deviner le nombre secret.

Deux situations sont possibles :

- Si le joueur B trouve le bon nombre, il remporte l'ensemble des fonds déposés dans le contrat.
- Si le joueur B échoue après le nombre maximal de tentatives, le joueur A récupère la totalité des ETH.

Le smart contract utilise des fonctions `payable` pour recevoir les mises et transfère automatiquement le solde au gagnant.

4 Version 3 : Smart Contract générique

Dans les versions précédentes, chaque partie nécessitait le déploiement d'un nouveau smart contract. Cette approche n'est pas idéale car le déploiement de contrats sur Ethereum est coûteux en gas. La version 3 introduit donc un smart contract générique capable de gérer plusieurs parties simultanément.

Chaque partie est représentée par une structure appelée **Game**, qui contient toutes les informations nécessaires :

1. les adresses des joueurs
2. le nombre secret
3. les mises en ETH
4. le nombre de tentatives
5. l'état de la partie

Les parties sont stockées dans une structure de type **mapping** associant un identifiant unique à chaque partie. Cette architecture permet à plusieurs paires de joueurs de jouer en parallèle sans avoir à déployer un nouveau contrat à chaque fois.

Deux interfaces Streamlit ont été développées :

- une interface pour le joueur A permettant de créer une partie
- une interface pour le joueur B permettant de rejoindre une partie et de faire des tentatives

5 Version 4 : Confidentialité du nombre (Commit–Reveal)

La version 4 introduit un mécanisme de sécurité important : la confidentialité du nombre secret.

Dans les versions précédentes, le nombre secret était stocké directement dans le smart contract. Or les données de la blockchain sont publiques, ce qui signifie qu'un utilisateur pourrait potentiellement consulter l'état du contrat et découvrir le nombre.

Pour résoudre ce problème, nous utilisons un mécanisme appelé **commit–reveal**.

Ce protocole se déroule en deux étapes :

5.1 Commit

Le joueur A calcule un hash cryptographique du nombre secret combiné avec une valeur aléatoire appelée *salt* :

$$\text{commitHash} = \text{keccak256}(\text{secretNumber}, \text{salt})$$

Seul ce hash est enregistré dans le smart contract.

5.2 Reveal

À la fin de la partie, le joueur A révèle le nombre secret et le salt. Le smart contract recalcule alors le hash et vérifie qu'il correspond au hash initial.

Cette technique garantit que le joueur A ne peut pas modifier le nombre secret pendant la partie.

Le contrat vérifie également que toutes les réponses données au joueur B étaient cohérentes avec le nombre secret.

Si une incohérence est détectée, le joueur B est automatiquement déclaré gagnant.

6 Architecture de l'application

L'application repose sur trois composants principaux : un smart contract en Solidity qui implémente la logique du jeu et gère les transactions, une application Python utilisant `web3.py` pour interagir avec la blockchain, et une interface Streamlit permettant aux joueurs d'utiliser facilement la DApp.

L'application Python se connecte au réseau Ethereum Sepolia via un endpoint RPC et envoie des transactions signées au smart contract. Les événements générés par le contrat sont ensuite récupérés afin d'afficher les résultats dans l'interface utilisateur.